

LAPORAN TUGAS BESAR 3

IF2211 STRATEGI ALGORITMA

DEADLINE: Sabtu, 30 Mei 2026 Pukul 23:59 WIB



Dipersiapkan oleh:

Kelompok Bet-ter

13524022 Ishak Palentino Napitupulu

13424060 Steven Tan

13524063 Marcel Luther Sitorus

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132 2026

Daftar Isi

Daftar Isi.....	1
BAB 1 DESKRIPSI TUGAS.....	2
Spesifikasi Wajib.....	2
Spesifikasi Bonus (Maks 20 poin).....	4
BAB 2 LANDASAN TEORI.....	6
2.1. Knuth-Morris-Pratt.....	6
2.2. Boyer-Moore.....	7
2.3. Aho-Corasick.....	8
2.4. Rabin Karp.....	10
2.5. Weighted Levenshtein Distance.....	11
2.6. Chromium Extension.....	12
BAB 3 ANALISIS PEMECAHAN MASALAH.....	13
3.1. Langkah - Langkah Pemecahan Masalah.....	13
3.2. Proses Pemetaan Knuth-Morris-Pratt.....	13
3.3. Proses Pemetaan Boyer-Moore.....	14
3.4. Fitur fungsional dan arsitektur extension yang dibangun.....	15
3.5. Contoh ilustrasi kasus.....	15
BAB 4 IMPLEMENTASI DAN PENGUJIAN.....	17
4.1. Implementasi.....	17
4.1.1. Implementasi Knuth-Morris-Pratti.....	17
4.1.2. Implementasi Boyer-Moore.....	19
4.1.3. Implementasi Aho-Corasick.....	21
4.1.4. Implementasi Rabin Karp.....	23
4.1.5. Implementasi Fuzzy Matching.....	24
4.2. Pengujian.....	28
4.2.1. Pengujian 1.....	28
4.2.2. Pengujian 2.....	32
4.2.3. Pengujian 3.....	35
4.2.4. Pengujian 4.....	42
4.2.5. Pengujian 5.....	49
BAB 5 KESIMPULAN DAN SARAN.....	56
5.1. Kesimpulan.....	56
5.2. Saran.....	56
LAMPIRAN.....	57
DAFTAR PUSTAKA.....	59

BAB 1

DESKRIPSI TUGAS

Spesifikasi Wajib

Pada tugas besar ini, buatlah sebuah *chromium browser extension* yang dapat mendeteksi informasi yang mengandung unsur judol. Sebagai catatan, seperti yang biasa kita temukan dalam website sekarang dengan format <kata><angka>. Contoh: GACOR99, MAXWIN88, MADU308, dan sebagainya. Sistem harus memiliki **minimal fitur** dengan detail seperti berikut.

1. Extension yang dibuat pada tugas besar ini dikembangkan dengan bahasa **Typescript**. Dalam proses pencocokan kata kunci, sistem wajib mengimplementasikan algoritma **Knuth-Morris-Pratt** dan **Boyer Moore** serta **Regex**. Untuk algoritma selain Regex, pencarian dilakukan menggunakan file [keyword.txt](#) (file ini boleh dimodifikasi sesuai kebutuhan) yang dipisahkan dengan newline untuk melakukan string matching secara iterative, sehingga pengguna tidak memasukkan input, tapi sistem yang membaca file tersebut secara langsung.
Untuk pencarian dengan regex, format pencarian **seminimal mungkin** dapat handle 2 atau 3 angka di belakang kata tanpa spasi. Contoh: MAXWIN234, SLOT99. Test case yang diberikan nanti akan sangat bervariasi, sehingga pastikan pencarian Regex diimplementasikan dengan detail dan dapat **handle banyak edge case**. *Adapun juga algoritma Aho-Corasick dan Rabin Karp apabila ingin mengerjakan bonus, sesuai tertera di Spesifikasi Bonus dibawah.*
2. Perlu diketahui bahwa, Developer Judol pun juga membuat penamaan dari websitenya, seperti pada teknik *Phising*, jadi membuat nama domain yang serupa dengan pada umumnya, namun nyatanya berbeda. Ini terjadi oleh karena adanya perbedaan kode komputer (*unicode*), dan komputer mengiranya adalah karakter yang berbeda. Contoh:

Gacor999	Gacor999
HOKI88	H0KI88
dan lain sebagainya	

Maka dari itu, sistem harus bisa membaca kecocokan secara exact matching dan juga fuzzy matching (apabila tidak ditemukan kecocokan yang persis). Untuk setiap kata kunci yang tidak memiliki kemunculan sama sekali pada saat *exact matching*, lakukan pencarian kembali dengan perhitungan tingkat kemiripan menggunakan algoritma [Weighted Levenshtein Distance](#). Penyesuaian bobot (*weight*) ini krusial karena pelaku sering kali menyiasati strategi dengan

menggunakan huruf pengganti yang mirip secara visual (misalnya mengubah huruf 'O' menjadi angka '0', 'A' menjadi '4', atau 'I' menjadi '1'). Implementasi harus memperhatikan hal-hal berikut:

1. **Weighted Cost:** Berikan bobot penalti yang lebih kecil untuk substitusi karakter yang mirip secara visual dibandingkan dengan karakter yang benar-benar berbeda.
2. **Thresholding:** Tentukan nilai ambang batas (*threshold*) kemiripan yang tepat untuk memastikan sistem tetap memberikan hasil yang relevan namun tetap menghindari munculnya *false positive*.

Algoritma ini memungkinkan sistem untuk tetap menampilkan hasil pencarian yang relevan meskipun terdapat perbedaan minor, kesalahan ketik, atau upaya manipulasi teks oleh pengguna. *Perlu diperhatikan bahwa secara konsep, exact matching juga merupakan bagian dari fuzzy matching.*

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ \text{lev}(\text{tail}(a), \text{tail}(b)) & \text{if } \text{head}(a) = \text{head}(b), \\ 1 + \min \begin{cases} \text{lev}(\text{tail}(a), b) \\ \text{lev}(a, \text{tail}(b)) \\ \text{lev}(\text{tail}(a), \text{tail}(b)) \end{cases} & \text{otherwise} \end{cases}$$

Weighted Levenshtein Distance Formula

3. Jadi sistem menjalankan pengecekan dengan hirarki sebagai berikut:
 - a. **Exact Matching:** Algoritma *pattern matching* untuk mencari kata kunci dasar dari keyword.txt
 - b. **RegEx Matching:** Secara paralel atau sekuensial, jalankan Regex untuk menangkap pola <kata><angka>
 - c. **Fuzzy Matching:** Kalau string dalam DOM tidak lolos namun memiliki kemiripan, jalankan dengan Weighted Levenshtein Distance dalam upaya manipulasi karakter
4. Apabila ditemukan result keyword, maka elemen DOM terkait harus ditandai, dengan warna bebas. Namun highlight tidak boleh merusak layout, tepat sasaran dan dapat terhapus saat rescanning.
5. Saat user hover pada elemen yang terdeteksi, maka tampilkan tooltip dengan informasi berikut:
 - a. Keyword yang terdeteksi
 - b. Algoritma yang digunakan
 - c. Jumlah kemunculan

- d. Waktu eksekusi
- Tooltip dibuat dengan custom DOM element.
- 6. Terdapat juga statistik secara realtime pada popup extension yang menampilkan:
 - a. Total keyword yang ditemukan
 - b. Perbandingan jumlah keyword yang satu dengan lainnya, menggunakan visualisasi
 - c. Waktu eksekusi tiap algoritma
 - d. Jumlah match tiap algoritma
 - 7. Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link [berikut](#), paling lambat **Minggu, 17 Mei 2026 23.59**. Nama kelompok dilarang mengandung unsur SARA, penghinaan, provokasi, atau hal-hal lain yang bertentangan dengan norma hukum.
Perlu diperhatikan bahwa kelompok tubes untuk mata kuliah ini **tidak boleh sama**. Jadi jika berkelompok dengan X pada tubes ini, maka pada tubes selanjutnya anda tidak diperkenankan berkelompok dengan X.

Perhatian

- a. Seluruh algoritma string matching wajib dibuat *from scratch*
- b. **Dilarang keras** menggunakan
 - i. *string.includes()*
 - ii. *string.indexOf()*
 - iii. *Built-in search function*
 - iv. *External string matching library*
- c. Implementasi manual wajib mencakup:
 - i. *Border function*
 - ii. *Failure function*
 - iii. *Last occurrence table*
 - iv. *Shifting process*
 - v. *Comparison Counting*
- d. Regex **diperbolehkan** menggunakan regex engine dari JavaScript

Spesifikasi Bonus (Maks 20 poin)

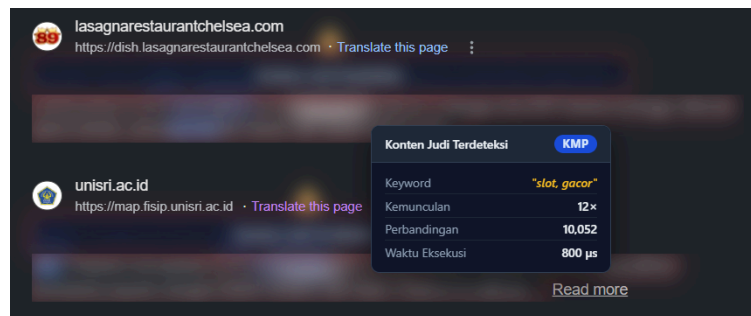
- 1. (6 poin) **[Video]** Membuat video tentang browser extension Judol Detector yang dikembangkan, kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya

dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. Semakin **menarik** dan **informatif** video, maka semakin banyak poin yang diberikan.

2. (4 poin) **[Algoritma Bonus]** Mengimplementasikan Algoritma [Aho-Corasick](#) dan [Rabin Karp](#) sebagai algoritma alternatif.

3. (5 poin) **[Censorship / Blur Teks]**

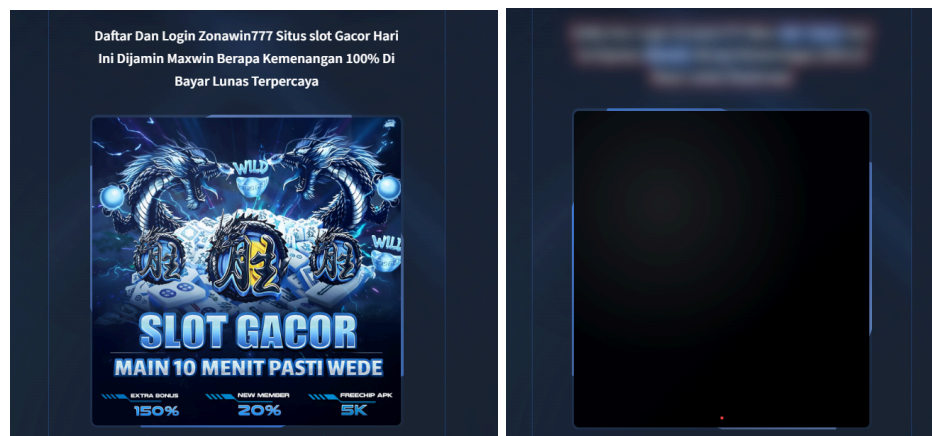
Mengimplementasikan fitur blur visual secara otomatis pada elemen halaman web yang terdeteksi memuat konten judi online. Tidak hanya sekedar highlight teks, ekstensi langsung melakukan aksi blur pada elemen yang mengandung kata kunci judol, sehingga konten tersebut tidak terlihat oleh pengguna. Perlu diperhatikan, terdapat toggle on/off untuk ini.



Gambar 3 : Contoh Blur Teks

4. (5 poin) **[Optical Character Recognition Pada Gambar]**

Mengimplementasikan fitur OCR untuk mendeteksi kata kunci judi online yang sengaja disembunyikan dalam bentuk gambar. Ekstensi akan mengekstrak teks dari elemen gambar yang ditemukan di halaman, kemudian mencocokkannya dengan daftar kata kunci judol, lalu melakukan blur atau penggantian pada gambar yang terdeteksi mengandung konten judol. Fitur ini dapat diimplementasikan dengan [Tesseract.js](#)



Gambar 4 : Contoh Hasil Sensor Gambar

Catatan: Gambar pengganti tidak harus hitam, bisa dikasih gambar apapun ^ _ ^

BAB 2

LANDASAN TEORI

2.1. Knuth-Morris-Pratt

Algoritma Knuth-Morris-Pratt atau KMP adalah algoritma pencocokan string yang digunakan untuk mencari kemunculan sebuah pola (*pattern*) di dalam suatu teks. Algoritma ini dikembangkan oleh Donald Knuth, Vaughan Pratt, dan James H. Morris. KMP dirancang untuk mengatasi kelemahan algoritma pencocokan string sederhana (*brute force*) yang sering melakukan perbandingan ulang terhadap karakter yang sebenarnya sudah pernah diperiksa.

Pada pencocokan string biasa, ketika terjadi ketidakcocokan antara karakter pada teks dan pola, posisi pencarian biasanya digeser satu karakter ke kanan, lalu proses perbandingan dimulai kembali dari awal pola. Hal ini dapat menyebabkan banyak perbandingan berulang, terutama jika pola memiliki banyak karakter yang sama atau berulang. Algoritma KMP menyelesaikan masalah ini dengan memanfaatkan informasi dari pola itu sendiri, sehingga ketika terjadi ketidakcocokan, pola dapat digeser secara lebih efisien tanpa perlu membandingkan ulang karakter yang sudah diketahui cocok.

Konsep utama dalam algoritma KMP adalah penggunaan tabel Longest Prefix Suffix atau LPS. Tabel LPS menyimpan panjang awalan terpanjang dari pola yang juga merupakan akhiran dari bagian pola tertentu. Awalan (*prefix*) adalah bagian awal dari string, sedangkan akhiran (*suffix*) adalah bagian akhir dari string. Dengan informasi ini, algoritma dapat menentukan posisi pola berikutnya yang masih mungkin cocok setelah terjadi ketidakcocokan.

Sebagai contoh, jika sebagian pola sudah cocok dengan teks lalu terjadi ketidakcocokan pada karakter berikutnya, algoritma KMP tidak langsung mengulang pencocokan dari awal pola. Sebaliknya, KMP melihat nilai LPS dari bagian pola yang sudah cocok untuk mengetahui berapa banyak karakter pada pola yang masih dapat dipertahankan. Dengan demikian, indeks pada teks tidak perlu mundur, dan pencarian dapat dilanjutkan secara lebih efisien.

Secara umum, algoritma KMP terdiri dari dua tahap utama. Tahap pertama adalah preprocessing, yaitu membangun tabel LPS berdasarkan pola yang akan dicari. Tahap kedua adalah searching, yaitu melakukan pencocokan pola terhadap teks dengan bantuan tabel LPS. Pada tahap pencarian, terdapat dua indeks yang digunakan, yaitu indeks untuk teks dan indeks untuk pola. Jika karakter pada teks dan pola cocok, kedua indeks akan maju. Jika tidak cocok, indeks pada pola akan disesuaikan berdasarkan nilai LPS, sementara indeks pada teks tidak perlu kembali ke posisi sebelumnya.

Keunggulan utama algoritma KMP terletak pada efisiensi waktunya. Proses pembentukan tabel LPS memiliki kompleksitas waktu $O(m)$, dengan m adalah panjang pola. Proses pencarian pola dalam teks memiliki kompleksitas waktu $O(n)$, dengan n adalah panjang teks. Dengan demikian,

kompleksitas total algoritma KMP adalah $O(n + m)$. Kompleksitas ini lebih baik dibandingkan metode brute force yang dalam kasus terburuk dapat mencapai $O(n \times m)$.

Dalam konteks pencocokan pola pada teks, algoritma KMP sangat cocok digunakan ketika pencarian harus dilakukan secara cepat dan pola yang dicari memiliki kemungkinan pengulangan karakter. Karena tidak melakukan perbandingan ulang yang tidak perlu, KMP dapat bekerja secara konsisten dan efisien bahkan pada teks yang panjang. Oleh karena itu, algoritma ini banyak digunakan dalam sistem pencarian teks, analisis string, deteksi kata kunci, dan berbagai aplikasi yang membutuhkan pencocokan pola secara tepat.

2.2. Boyer-Moore

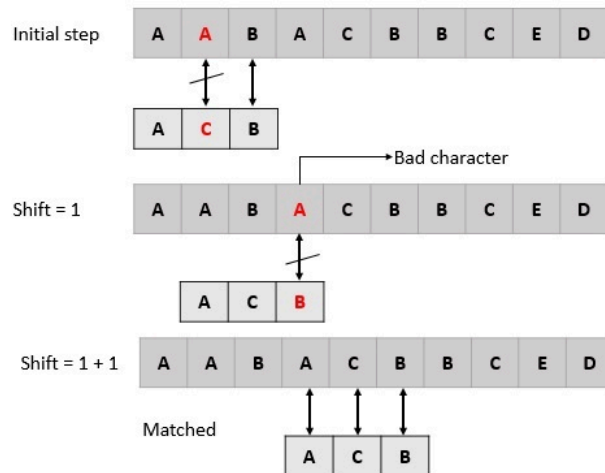
Algoritma Boyer-Moore merupakan algoritma pencocokan string yang efisien dan cepat dalam menemukan pola dalam teks yang panjang. Algoritma ini dirancang oleh Robert S. Boyer dan J. Strother Moore pada tahun 1977 dan masih digunakan hingga saat ini karena kompleksitas waktu pencariannya jauh lebih tinggi dibandingkan dengan algoritma pencarian linear.

Prinsip kerja Boyer-Moore berbeda dari algoritma pencarian lainnya karena memeriksa pola dari kanan ke kiri (right-to-left scan) dan melibatkan dua teknik utama heuristik. Teknik pertama adalah Bad Character Rule, yaitu jika karakter pola tidak cocok dengan karakter teks, maka pola digeser berdasarkan posisi karakter yang tidak cocok tersebut. Teknik kedua adalah Good Suffix Rule, yaitu jika bagian akhir (suffix) dari pola cocok tetapi ada karakter yang tidak cocok, maka pola digeser berdasarkan suffix yang cocok ters

Algoritma Boyer-Moore merupakan algoritma pencocokan string yang efisien dan cepat dalam menemukan pola dalam teks yang panjang. Algoritma ini dirancang oleh Robert S. Boyer dan J. Strother Moore pada tahun 1977 dan masih digunakan hingga saat ini karena kompleksitas waktu pencariannya jauh lebih tinggi dibandingkan dengan algoritma pencarian linear.

Prinsip kerja Boyer-Moore berbeda dari algoritma pencarian lainnya karena memeriksa pola dari kanan ke kiri (right-to-left scan) dan melibatkan dua teknik utama heuristik. Teknik pertama adalah Bad Character Rule, yaitu jika karakter pola tidak cocok dengan karakter teks, maka pola digeser berdasarkan posisi karakter yang tidak cocok tersebut. Teknik kedua adalah Good Suffix Rule, yaitu jika bagian akhir (suffix) dari pola cocok tetapi ada karakter yang tidak cocok, maka pola digeser berdasarkan suffix yang cocok tersebut.

Kombinasi kedua aturan ini membuat algoritma Boyer-Moore jauh lebih efisien dibandingkan pencarian linear yang memeriksa teks dari kiri ke kanan



Gambar 2.2.1 Langkah pencocokan *string* dengan *boyer-moore*

Tahapan utama algoritma Boyer-Moore dalam melakukan pencocokan string adalah:

1. Inisialisasi dengan mendefinisikan teks (T) dan pola (P) serta menentukan posisi awal perbandingan,
2. Preprocessing untuk membangun tabel heuristik bad character dan good suffix berdasarkan pola yang diberikan,
3. Pencocokan dengan membandingkan pattern dengan teks dari kanan ke kiri, dan
4. Pergeseran dengan menggerakkan pattern berdasarkan nilai terbesar dari pergeseran OH (Occurrence Heuristic) dan MH (Match Heuristic).

Algoritma Boyer-Moore memiliki empat konsep dasar dalam proses pencarian string yaitu preprocessing, right-to-left scan, bad-character rule, dan good-suffix rule. Precomputation dari algoritma ini terdiri dari bad-character preprocessing dan good-suffix preprocessing. Dengan tahapan ini, Boyer-Moore mampu mengurangi jumlah perbandingan karakter secara signifikan dibandingkan metode Brute Force, sehingga cocok digunakan dalam sistem pencarian data seperti katalog produk digital, sistem layanan berbasis web, aplikasi kamus, dan sistem penelusuran informasi. Algoritma ini merupakan algoritma pencarian string paling efektif dan menjadi standar untuk berbagai literatur pencarian string.

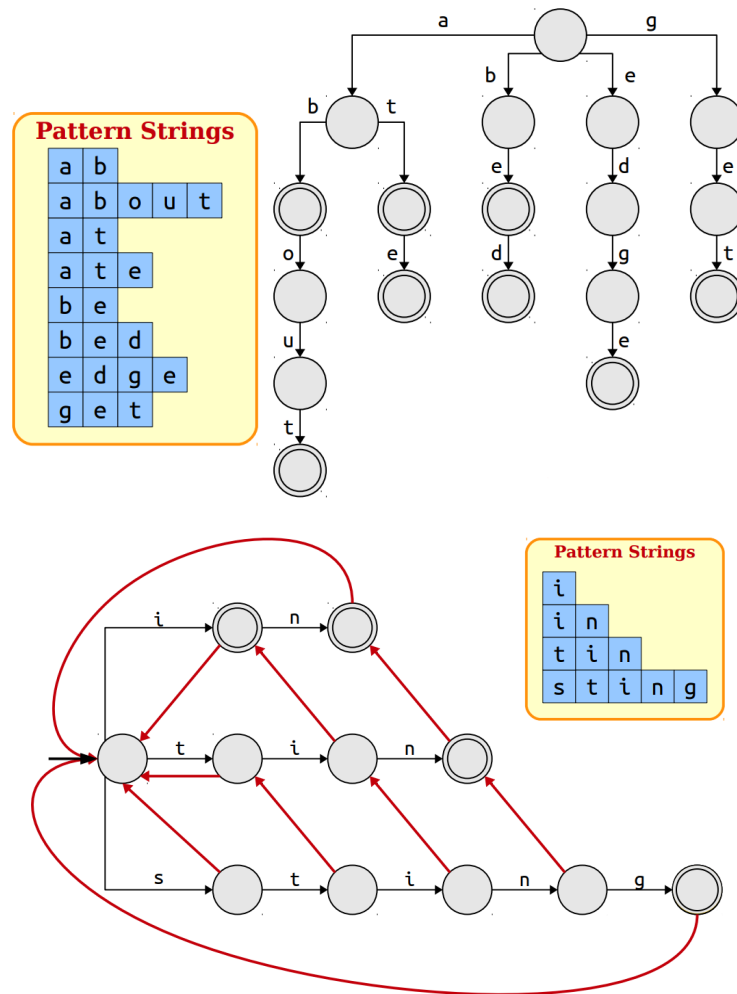
2.3. Aho-Corasick

Algoritma Aho-Corasick adalah algoritma pencarian string yang ditemukan oleh Alfred V. Aho dan Margaret J. Corasick pada tahun 1975. Algoritma ini mampu mendeteksi beberapa pola (multipattern) secara bersamaan dalam sebuah teks dengan membangun struktur trie (prefix tree) yang dilengkapi failure links untuk transisi cepat saat terjadi ketidakcocokan.

Aho-Corasick automaton

Algoritma Aho-Corasick adalah algoritma pencarian string yang ditemukan oleh Alfred V. Aho dan Margaret J. Corasick pada tahun 1975. Algoritma ini mampu mendeteksi beberapa pola

(multipattern) secara bersamaan dalam sebuah teks dengan membangun struktur trie (prefix tree) yang dilengkapi failure links untuk transisi cepat saat terjadi ketidakcocokan.



Gambar 2.3.1 Trie dengan tanpa dan dengan *Failure Route*

Proses kerja algoritma terdiri dari dua tahap:

1. konstruksi trie dari kumpulan pola yang akan dicari, dan
2. penambahan failure links yang mengarah ke node dengan sufiks terpanjang yang juga merupakan prefix dari pola dalam kamus. Algoritma ini memiliki kompleksitas waktu $O(m + n + z)$, di mana n adalah panjang teks, m adalah total panjang semua pola, dan z adalah jumlah kecocokan yang ditemukan.

Keunggulan utama algoritma ini adalah kemampuan melakukan pencarian *multipattern* dalam satu kali proses tanpa perlu memundurkan pointer pada teks (no backtracking), sehingga lebih efisien dibandingkan mencari setiap pola satu per satu. Algoritma ini banyak digunakan dalam utilitas grep, Intrusion Prevention System (IPS), sistem deteksi intrusi jaringan, dan aplikasi pencarian informasi.

2.4. Rabin Karp

Algoritma Rabin-Karp adalah salah satu algoritma untuk mencari suatu kata atau pola di dalam teks. Misalnya, jika terdapat teks panjang dari halaman web, lalu sistem ingin mencari kata seperti slot, gacor, atau maxwin, maka algoritma Rabin-Karp dapat digunakan untuk mencari apakah kata-kata tersebut muncul di dalam teks.

Ide utama dari Rabin-Karp adalah menggunakan nilai hash. Nilai hash dapat dianggap seperti “sidik jari” dari sebuah string. Sebuah kata akan diubah menjadi angka tertentu. Jika dua string memiliki nilai hash yang berbeda, maka kedua string tersebut pasti berbeda. Karena itu, algoritma tidak perlu membandingkan semua karakter satu per satu pada setiap posisi.

Sebagai contoh, misalkan pola yang dicari adalah SLOT. Algoritma akan menghitung nilai hash dari kata SLOT. Setelah itu, teks halaman akan diperiksa menggunakan jendela pencarian dengan panjang yang sama, yaitu 4 karakter. Jendela ini akan digeser satu per satu dari kiri ke kanan. Setiap potongan teks dengan panjang 4 karakter akan dihitung nilai hash-nya, lalu dibandingkan dengan nilai hash dari SLOT.

Contoh sederhana:

```
Teks      : SITUS SLOT GACOR
Pola      : SLOT
```

```
Jendela 1 : SITU
Jendela 2 : ITUS
Jendela 3 : TUS
Jendela 4 : US S
Jendela 5 : S SL
Jendela 6 : SLOT → cocok
```

Pada contoh tersebut, Rabin-Karp tidak langsung membandingkan seluruh karakter pada setiap jendela. Algoritma terlebih dahulu membandingkan nilai hash-nya. Jika nilai hash berbeda, maka jendela tersebut langsung dilewati. Jika nilai hash sama, barulah dilakukan pengecekan karakter untuk memastikan apakah kata tersebut benar-benar sama.

Bagian penting dari Rabin-Karp adalah rolling hash. Rolling hash adalah cara untuk memperbarui nilai hash ketika jendela pencarian bergeser. Tanpa rolling hash, setiap jendela harus dihitung ulang dari awal. Dengan rolling hash, nilai hash jendela baru dapat dihitung dari nilai hash jendela sebelumnya. Caranya adalah dengan membuang karakter paling kiri, menggeser sisa karakter, lalu menambahkan karakter baru di sebelah kanan.

Sebagai contoh:

```
Jendela lama : SITU
Jendela baru : ITUS
```

Ketika jendela bergeser, karakter S di kiri dibuang, lalu karakter S baru di kanan dimasukkan. Dengan cara ini, proses perhitungan menjadi lebih efisien karena algoritma tidak perlu menghitung semua karakter dari awal setiap kali jendela bergeser.

Meskipun menggunakan hash, Rabin-Karp tetap perlu melakukan pengecekan karakter ketika nilai hash sama. Hal ini karena ada kemungkinan dua string yang berbeda menghasilkan nilai hash yang sama. Kejadian ini disebut collision. Oleh karena itu, nilai hash hanya digunakan sebagai pengecekan awal. Jika hash sama, algoritma tetap membandingkan karakter satu per satu untuk memastikan bahwa hasilnya benar-benar cocok.

Secara umum, Rabin-Karp bekerja cukup efisien. Pada kasus rata-rata, kompleksitas waktunya adalah $O(n + m)$, dengan n sebagai panjang teks dan m sebagai panjang pola. Namun, pada kasus terburuk, jika terlalu banyak collision, kompleksitasnya dapat menjadi $O(nm)$ karena algoritma harus sering melakukan pengecekan karakter ulang.

2.5. Weighted Levenshtein Distance

Weighted Levenshtein adalah pengembangan dari algoritma Levenshtein Distance yang digunakan untuk mengukur tingkat perbedaan antara dua string. Pada Levenshtein Distance biasa, perbedaan antara dua string dihitung berdasarkan jumlah minimum operasi edit yang diperlukan untuk mengubah satu string menjadi string lainnya. Operasi edit tersebut terdiri dari insertion atau penyisipan karakter, deletion atau penghapusan karakter, dan substitution atau penggantian karakter.

Pada Levenshtein Distance standar, setiap operasi edit biasanya memiliki bobot yang sama, yaitu bernilai 1. Artinya, menghapus satu karakter, menambahkan satu karakter, atau mengganti satu karakter dianggap memiliki tingkat kesalahan yang sama. Namun, dalam beberapa kasus, tidak semua kesalahan memiliki tingkat kepentingan yang sama. Misalnya, kesalahan pada huruf, angka, simbol, atau spasi dapat dianggap memiliki dampak yang berbeda. Oleh karena itu, digunakan algoritma Weighted Levenshtein yang memungkinkan setiap operasi edit memiliki bobot atau biaya yang berbeda.

Konsep utama dari Weighted Levenshtein adalah menghitung jarak minimum antara dua string dengan mempertimbangkan biaya khusus untuk setiap operasi edit. Operasi insertion memiliki biaya ketika sebuah karakter perlu ditambahkan, operasi deletion memiliki biaya ketika sebuah karakter perlu dihapus, dan operasi substitution memiliki biaya ketika sebuah karakter diganti dengan karakter lain. Bobot tersebut dapat ditentukan berdasarkan kebutuhan sistem atau karakteristik data yang diproses.

Sebagai contoh, jika sistem menganggap kesalahan pada spasi atau simbol lebih ringan dibandingkan kesalahan pada huruf atau angka, maka operasi terhadap spasi dan simbol dapat diberi bobot lebih kecil. Sebaliknya, jika perubahan pada angka dianggap lebih penting karena dapat mengubah makna secara signifikan, maka biaya substitusi atau penghapusan angka dapat dibuat lebih besar. Dengan cara ini, Weighted Levenshtein dapat memberikan hasil pencocokan yang lebih fleksibel dan sesuai konteks dibandingkan Levenshtein Distance biasa.

Perhitungan Weighted Levenshtein umumnya dilakukan menggunakan pendekatan dynamic programming. Dua string yang dibandingkan disusun dalam bentuk matriks, dengan baris mewakili karakter dari string pertama dan kolom mewakili karakter dari string kedua. Setiap sel pada matriks menyimpan biaya minimum yang dibutuhkan untuk mengubah sebagian string pertama menjadi sebagian string kedua. Nilai pada setiap sel dihitung berdasarkan kemungkinan operasi insertion, deletion, dan substitution, lalu dipilih nilai dengan biaya terkecil.

Secara umum, rumus perhitungan Weighted Levenshtein dapat dinyatakan sebagai berikut. Jika dua karakter yang dibandingkan sama, maka tidak ada biaya tambahan. Namun, jika karakter berbeda, maka biaya substitusi ditambahkan sesuai bobot yang telah ditentukan. Selain itu, algoritma juga mempertimbangkan biaya penyisipan dan penghapusan karakter. Nilai minimum dari ketiga kemungkinan operasi tersebut menjadi nilai jarak pada posisi matriks tersebut.

Keunggulan utama Weighted Levenshtein adalah kemampuannya menyesuaikan proses pencocokan string berdasarkan tingkat kesalahan yang dianggap penting. Algoritma ini berguna untuk pencarian teks yang toleran terhadap kesalahan penulisan, variasi penulisan, atau manipulasi karakter tertentu. Dengan memberikan bobot berbeda pada jenis karakter atau jenis operasi, sistem dapat menghasilkan penilaian kemiripan yang lebih akurat sesuai kebutuhan aplikasi.

Dalam konteks pencocokan pola, Weighted Levenshtein dapat digunakan untuk mendeteksi pola yang tidak harus sama persis dengan kata kunci, tetapi masih memiliki tingkat kemiripan tertentu. Hal ini berguna ketika teks yang diproses memiliki kemungkinan typo, penggunaan simbol tambahan, variasi spasi, atau perubahan karakter. Oleh karena itu, Weighted Levenshtein sering digunakan dalam sistem pencarian fuzzy, koreksi ejaan, deteksi kemiripan string, dan analisis teks yang membutuhkan pencocokan fleksibel.

2.6. Chromium Extension

Chromium extension adalah program tambahan yang berjalan pada browser berbasis Chromium, seperti Google Chrome, Microsoft Edge, Brave, dan Opera. Extension digunakan untuk menambahkan fitur baru, mengubah perilaku browser, atau membantu pengguna melakukan aktivitas tertentu secara lebih praktis.

Sebuah Chromium extension umumnya terdiri dari beberapa komponen, seperti manifest file, content script, background script, dan tampilan antarmuka seperti popup. File manifest berfungsi sebagai konfigurasi utama yang berisi informasi extension, izin akses, serta daftar script yang digunakan. Content script digunakan untuk berinteraksi langsung dengan halaman web, sedangkan background script menangani proses yang berjalan di belakang layar.

Dalam pengembangan aplikasi, Chromium extension dapat digunakan untuk membaca atau memproses konten halaman web, menambahkan elemen pada tampilan browser, memblokir konten tertentu, atau melakukan deteksi otomatis terhadap teks pada halaman. Karena dapat berinteraksi dengan halaman web yang sedang dibuka, extension harus menggunakan izin akses secara hati-hati agar tetap aman dan tidak melanggar privasi pengguna.

BAB 3

ANALISIS PEMECAHAN MASALAH

3.1. Langkah - Langkah Pemecahan Masalah

Permasalahan utama pada tugas ini adalah mendeteksi konten yang mengandung unsur judi online pada halaman web. Konten tersebut dapat muncul dalam bentuk teks biasa, pola gabungan kata dan angka, variasi penulisan yang dimanipulasi, maupun teks yang berada di dalam gambar. Oleh karena itu, masalah tidak hanya diselesaikan dengan satu jenis pencarian, tetapi dibagi menjadi beberapa tahap.

Tahap pertama adalah menentukan sumber data yang akan diperiksa. Sistem mengambil teks dari DOM halaman web sebagai sumber utama. Selain itu, jika fitur OCR diaktifkan, sistem juga mengambil teks dari gambar yang terdapat pada halaman. Dengan demikian, input sistem dapat berasal dari dua sumber, yaitu teks halaman dan teks hasil OCR.

Tahap kedua adalah menentukan pola yang akan dicari. Untuk exact matching, pola berasal dari daftar kata kunci pada file keyword.txt. Untuk pola seperti SLOT99 atau MAXWIN234, sistem menggunakan RegEx karena bentuk tersebut memiliki pola umum berupa gabungan kata dan angka. Untuk teks yang dimanipulasi, seperti penggunaan angka 0 sebagai pengganti huruf O, sistem menggunakan pendekatan fuzzy matching.

Tahap ketiga adalah menjalankan algoritma pencocokan. Teks yang telah dikumpulkan diproses oleh beberapa algoritma, yaitu Knuth-Morris-Pratt, Boyer-Moore, RegEx, Weighted Levenshtein, serta algoritma bonus jika diaktifkan. Setiap algoritma menghasilkan informasi berupa kata yang cocok, posisi kecocokan, jumlah match, jumlah perbandingan, dan waktu eksekusi.

Tahap terakhir adalah menampilkan hasil kepada pengguna. Teks yang terdeteksi diberi highlight atau blur pada halaman. Ketika pengguna mengarahkan kursor ke hasil deteksi, sistem menampilkan tooltip berisi informasi deteksi. Popup extension juga menampilkan ringkasan hasil, seperti total keyword yang ditemukan, total match, dan waktu eksekusi tiap algoritma.

3.2. Proses Pemetaan Knuth-Morris-Pratt

Pada analisis ini, algoritma Knuth-Morris-Pratt digunakan untuk menyelesaikan bagian exact matching, yaitu mencari kata kunci dari keyword.txt di dalam teks halaman web. Masalah pencarian konten judi online dipetakan ke dalam elemen KMP sebagai berikut:

- Text → teks dari DOM halaman web atau hasil OCR
- Pattern → setiap keyword dari keyword.txt
- LPS → tabel bantuan untuk menghindari pencocokan ulang dari awal
- Output → posisi kemunculan keyword pada teks

Pada KMP, setiap keyword diperlakukan sebagai pattern. Sebelum pencarian dilakukan, sistem membentuk tabel Longest Prefix Suffix atau LPS dari pattern tersebut. Tabel ini digunakan ketika

terjadi ketidakcocokan karakter. Dengan adanya LPS, algoritma dapat menentukan posisi pattern berikutnya tanpa harus mengulang seluruh proses pencocokan dari awal.

Sebagai contoh, jika keyword yang dicari adalah gacor, maka teks halaman akan diperiksa dari kiri ke kanan. Jika sebagian karakter sudah cocok tetapi kemudian terjadi mismatch, KMP menggunakan tabel LPS untuk menentukan bagian pattern yang masih mungkin cocok. Dengan cara ini, proses pencarian menjadi lebih efisien dibandingkan pencarian brute force yang selalu mengulang dari awal.

Dalam sistem extension, hasil dari KMP berupa daftar kecocokan yang berisi keyword, teks yang cocok, posisi awal, dan posisi akhir. Hasil tersebut kemudian digunakan untuk menandai teks pada halaman dan menampilkan statistik pada popup.

3.3. Proses Pemetaan Boyer-Moore

Boyer-Moore juga digunakan untuk menyelesaikan exact matching, yaitu mencari keyword dari keyword.txt di dalam teks halaman. Namun, strategi Boyer-Moore berbeda dari KMP. Jika KMP mencocokkan pattern dari kiri ke kanan, Boyer-Moore mencocokkan pattern dari kanan ke kiri.

Masalah pencarian pada extension dipetakan ke dalam elemen Boyer-Moore sebagai berikut:

- Text → teks dari DOM halaman web atau hasil OCR
- Pattern → setiap keyword dari keyword.txt
- Last occurrence table → tabel posisi terakhir setiap karakter pada pattern
- Bad character rule → aturan untuk menentukan besar pergeseran saat mismatch
- Output → posisi kemunculan keyword pada teks

Sebelum pencarian dilakukan, Boyer-Moore membentuk tabel last occurrence. Tabel ini menyimpan posisi terakhir setiap karakter yang muncul pada pattern. Ketika terjadi mismatch, algoritma melihat karakter pada teks yang menyebabkan mismatch, lalu menggunakan tabel last occurrence untuk menentukan seberapa jauh pattern dapat digeser.

Sebagai contoh, jika keyword yang dicari adalah slot, Boyer-Moore akan mulai membandingkan dari karakter paling kanan pattern. Jika terjadi mismatch, pattern dapat digeser lebih jauh berdasarkan karakter yang ditemukan pada teks. Dengan strategi ini, Boyer-Moore sering kali dapat melewati beberapa posisi sekaligus, sehingga pencarian dapat menjadi lebih cepat pada kasus tertentu.

Dalam sistem extension, Boyer-Moore menghasilkan format keluaran yang sama dengan KMP, yaitu daftar match beserta posisi dan informasi eksekusinya. Hal ini membuat hasil KMP dan Boyer-Moore dapat dibandingkan pada popup berdasarkan jumlah match, waktu eksekusi, dan jumlah perbandingan.

3.4. Fitur fungsional dan arsitektur extension yang dibangun.

Extension yang dibangun memiliki beberapa fitur utama, yaitu pemindaian teks halaman, pencocokan menggunakan beberapa algoritma, highlight hasil deteksi, blur konten yang terdeteksi, tooltip informasi, statistik pada popup, serta OCR untuk membaca teks pada gambar.

Arsitektur sistem dibagi menjadi beberapa komponen. Popup berfungsi sebagai antarmuka pengguna untuk menjalankan scan, clear, mengatur fitur, dan melihat statistik. Content script berjalan pada halaman web untuk membaca DOM, menjalankan pemindaian, dan menandai hasil deteksi. Modul pageScanner berperan sebagai pengatur alur utama, mulai dari memuat keyword, mengumpulkan teks, menjalankan algoritma, hingga mengembalikan hasil ke popup. Modul textScanEngine bertanggung jawab menjalankan algoritma pencocokan string.

Untuk fitur OCR, sistem menggunakan alur terpisah. Content script mengirim permintaan OCR ke background service worker. Background kemudian meneruskannya ke offscreen document, tempat Tesseract dijalankan untuk membaca teks dari gambar. Teks hasil OCR kemudian diproses kembali menggunakan algoritma pencocokan string yang sama seperti teks DOM.

Secara sederhana, alur utama extension adalah:

```
Popup → Content Script → Page Scanner → Text Scan Engine →  
Hasil Deteksi
```

Sedangkan alur OCR adalah:

```
Content Script → Background Service Worker → Offscreen  
Document → Tesseract OCR → Text Scan Engine
```

Pembagian ini membuat setiap komponen memiliki tanggung jawab yang jelas. Popup berfokus pada tampilan, content script berfokus pada interaksi dengan halaman web, textScanEngine berfokus pada algoritma, dan OCR berfokus pada ekstraksi teks dari gambar.

3.5. Contoh ilustrasi kasus

Sebagai contoh, misalkan terdapat teks pada halaman web sebagai berikut:

```
Situs slot gacor99 dengan bonus maxwin234 dan H0KI88.
```

Pada teks tersebut, kata slot dapat dideteksi oleh exact matching menggunakan KMP dan Boyer-Moore karena kata tersebut dapat berada dalam daftar keyword. Pada teks gacor99, sistem dapat mendeteksi gacor sebagai exact matching, sedangkan RegEx dapat mendeteksi gacor99 sebagai pola gabungan kata dan angka. Teks maxwin234 juga dapat ditangani oleh RegEx karena memiliki bentuk <kata><angka>.

Untuk teks H0KI88, sistem dapat menggunakan fuzzy matching karena karakter 0 menyerupai huruf O. Dengan Weighted Levenshtein, sistem dapat menghitung kemiripan antara H0KI dan bentuk asli seperti HOKI.

Jika pada halaman terdapat gambar yang berisi tulisan SLOT GACOR, fitur OCR akan mengekstrak teks dari gambar tersebut. Hasil OCR kemudian diproses kembali oleh algoritma pencocokan string. Jika ditemukan keyword, gambar dapat diberi efek blur dan hasilnya ditampilkan pada popup.

Dari contoh ini, terlihat bahwa sistem tidak hanya mencari kata yang sama persis, tetapi juga menangani pola kata-angka, variasi penulisan yang dimanipulasi, serta teks yang tersembunyi di dalam gambar.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1. Implementasi

Repository ini memiliki kerangka sebagai ekstensi Chromium berbasis TypeScript yang terdiri dari popup sebagai antarmuka pengguna, content script sebagai pemindai halaman, modul matching sebagai penghubung algoritma, modul algoritma sebagai inti pattern matching, serta modul OCR untuk membaca teks dari gambar. Alur kerjanya dimulai ketika popup dibuka, kemudian popup mengirim perintah scan ke content script pada tab aktif. Content script mengambil teks dari DOM, memuat daftar keyword, menjalankan algoritma KMP, Boyer-Moore, Regex, Weighted Levenshtein, serta algoritma tambahan Rabin-Karp dan Aho-Corasick jika diaktifkan. Hasil deteksi digunakan untuk memberi highlight atau blur pada teks, membangun statistik, dan mengirimkan ringkasan kembali ke popup. Jika OCR diaktifkan, gambar pada halaman juga diproses menggunakan Tesseract.js melalui background service worker dan offscreen document, kemudian teks hasil OCR discan dengan algoritma yang sama.

4.1.1. Implementasi Knuth-Morris-Pratti

```
export function searchKMP(  
  keywords: string[],  
  text: string,  
  sourceType: MatchSource,  
  caseInsensitive: boolean,  
) : AlgorithmResult {  
  const start = performance.now();  
  const matches: MatchResult[] = [];  
  let comparisonCount = 0;  
  const searchableText = caseInsensitive ? text.toLowerCase() : text;  
  
  for (const keyword of keywords) {  
    if (keyword.length === 0) {  
      continue;  
    }  
  
    const searchableKeyword = caseInsensitive ?  
keyword.toLowerCase() : keyword;  
    const keywordLength = searchableKeyword.length;  
    const longestPrefixSuffix =  
buildLongestPrefixSuffix(searchableKeyword);
```

```

let textIndex = 0;
let keywordIndex = 0;

while (textIndex < searchableText.length) {
    comparisonCount += 1;

    if (searchableKeyword[keywordIndex] ===
searchableText[textIndex]) {
        textIndex += 1;
        keywordIndex += 1;

        if (keywordIndex === keywordLength) {
            const matchStart = textIndex - keywordLength;
            matches.push(
                matchResultGenerator(
                    keyword,
                    text.slice(matchStart, textIndex),
                    matchStart,
                    textIndex - 1,
                ),
            );
            keywordIndex = longestPrefixSuffix[keywordIndex -
1];
        }

        continue;
    }

    if (keywordIndex > 0) {
        keywordIndex = longestPrefixSuffix[keywordIndex - 1];
    } else {
        textIndex += 1;
    }
}

return algorithmResultGenerator(
    matches,
    performance.now() - start,

```

```

        comparisonCount,
        "KMP",
        sourceType,
    );
}

```

Implementasi KMP terdapat pada file:

src/algorithms/kmp.ts

Algoritma ini digunakan untuk mencari setiap keyword di dalam teks secara exact matching. Sebelum proses pencarian, kode membangun tabel Longest Prefix Suffix melalui fungsi:

buildLongestPrefixSuffix(keyword)

Tabel ini menyimpan informasi panjang prefix yang juga merupakan suffix pada keyword. Tujuannya agar ketika terjadi mismatch, pencarian tidak perlu kembali ke awal keyword, tetapi cukup melompat ke posisi yang sudah diketahui dari tabel LPS.

Alur implementasinya:

1. Teks dan keyword diubah ke lowercase jika caseInsensitive bernilai true.
2. Untuk setiap keyword, dibuat tabel LPS.
3. Dua indeks digunakan:
4. textIndex untuk posisi di teks,
5. keywordIndex untuk posisi di keyword.
6. Jika karakter cocok, kedua indeks maju.
7. Jika seluruh keyword cocok, hasil match disimpan ke dalam matches.
8. Jika mismatch, keywordIndex digeser berdasarkan tabel LPS.
9. Hasil akhir dikembalikan dalam bentuk AlgorithmResult.

KMP pada kode ini menghasilkan data seperti keyword yang cocok, teks yang cocok, posisi awal, posisi akhir, waktu eksekusi, dan jumlah perbandingan karakter.

4.1.2. Implementasi Boyer-Moore

```

export function searchBoyerMoore(
    keywords: string[],
    text: string,
    sourceType: MatchSource,
    caseInsensitive: boolean,
): AlgorithmResult {
    const startTime = performance.now();
    const matches: MatchResult[] = [];
}

```



```

let comparisonCount = 0;
const searchableText = caseInsensitive ? text.toLowerCase() : text;

for (const keyword of keywords) {
    const searchableKeyword = caseInsensitive ? keyword.toLowerCase() :
keyword;
    const result = searchKeyword(text, searchableText, keyword,
searchableKeyword);

    matches.push(...result.matches);
    comparisonCount += result.comparisonCount;
}

return {
    matches,
    executionTimeMs: performance.now() - startTime,
    comparisonCount,
    algorithm: "Boyer-Moore",
    source: sourceType,
};
}

```

Implementasi Boyer-Moore terdapat pada file:

src/algorithms/boyerMoore.ts

Pada kode ini, Boyer-Moore menggunakan pendekatan bad character rule melalui tabel kemunculan terakhir karakter, yaitu:

buildLastOccurrenceTable(keyword)

Tabel tersebut menyimpan posisi terakhir setiap karakter dalam keyword. Saat terjadi mismatch, algoritma dapat menggeser keyword lebih jauh berdasarkan posisi karakter yang salah tersebut.

Alur implementasinya:

1. Teks dan keyword dinormalisasi ke lowercase jika caseInsensitive aktif.
2. Untuk setiap keyword, fungsi searchKeyword() dijalankan.
3. Pencocokan dilakukan dari kanan ke kiri, mulai dari karakter terakhir keyword.
4. Jika semua karakter cocok, match disimpan.
5. Jika terjadi mismatch, karakter yang gagal dicocokkan disebut badCharacter.
6. Keyword digeser menggunakan rumus:
7. $\text{Math.max}(1, \text{keywordIndex} - \text{badCharacterLastIndex})$

4.1.3. Implementasi Aho-Corasick

```
export function searchAhoCorasick (  
  keywords: string[],  
  text: string,  
  sourceType: MatchSource,  
  caseInsensitive: boolean  
): AlgorithmResult  
{  
  let start = performance.now();  
  let finRes: MatchResult[] = [];  
  let comparisonCount = 0;  
  
  let sText: string = text;  
  let cKeywords: string[] = keywords.slice();  
  if (caseInsensitive)  
  {  
    sText = sText.toLowerCase();  
    cKeywords = keywords.map((keyword: string): string =>  
keyword.toLowerCase());  
  }  
  
  let trie = buildAutomaton(cKeywords);  
  let matchesByKeyword: number[][] = [];  
  let keyIdx = 0;  
  for (keyIdx = 0; keyIdx < cKeywords.length; keyIdx++)  
  {  
    matchesByKeyword.push([]);  
  }  
  
  let currentNode = 0;  
  let tIdx = 0;  
  let t1 = sText.length;  
  
  while (tIdx < t1)  
  {  
    let currentChar = sText[tIdx];
```

```

        while (currentNode !== 0 &&
trie[currentNode].children.get(currentChar) === undefined)
        {
            currentNode = trie[currentNode].fail;
            ++comparisonCount;
        }

        ++comparisonCount;
        let nextNode = trie[currentNode].children.get(currentChar);
        if (nextNode !== undefined)
        {
            currentNode = nextNode;
        }
        else
        {
            currentNode = 0;
        }

        for (let matchedKeywordIdx of trie[currentNode].outputs)
        {
            let keyl = cKeywords[matchedKeywordIdx].length;
            matchesByKeyword[matchedKeywordIdx].push(tIdx - keyl + 1);
        }

        ++tIdx;
    }

    for (keyIdx = 0; keyIdx < cKeywords.length; keyIdx++)
    {
        let keyl = cKeywords[keyIdx].length;
        for (let r of matchesByKeyword[keyIdx])
        {
            finRes.push
            (
                matchResultGenerator
                (
                    keywords[keyIdx],
                    text.slice(r, r + keyl),
                    r,

```

```

        r + keyl - 1
    )
    )
}

let end = performance.now();
let algoRes: AlgorithmResult = (
    algorithmResultGenerator
    (
        finRes,
        end - start,
        comparisonCount,
        "Aho-Corasick",
        sourceType
    )
)
return algoRes;
}

```

Implementasi Aho-Corasick pada repository ini digunakan untuk melakukan pencarian banyak keyword sekaligus dalam satu kali pembacaan teks. Berbeda dengan KMP dan Boyer-Moore yang pada umumnya memproses keyword satu per satu, Aho-Corasick membangun sebuah struktur automaton berbasis trie dari seluruh daftar keyword. Setiap node dalam trie menyimpan transisi karakter ke node berikutnya, failure link, dan daftar output keyword yang berakhir pada node tersebut. Pada tahap awal, semua keyword dimasukkan ke dalam trie karakter demi karakter. Setelah trie terbentuk, algoritma membangun failure link menggunakan proses breadth-first search. Failure link ini digunakan ketika terjadi ketidakcocokan karakter, sehingga pencarian dapat berpindah ke node lain yang masih mungkin menjadi awalan dari keyword lain tanpa harus mengulang dari awal. Selain itu, output dari node failure juga digabungkan ke node saat ini agar keyword yang saling tumpang tindih tetap dapat terdeteksi. Pada tahap pencarian, teks dibaca dari kiri ke kanan satu kali saja. Setiap karakter akan digunakan untuk berpindah antar-node pada automaton. Jika tidak ada transisi yang sesuai, algoritma mengikuti failure link sampai menemukan transisi yang cocok atau kembali ke root. Ketika node yang sedang aktif memiliki output, berarti terdapat satu atau lebih keyword yang berhasil ditemukan, lalu posisi awal dan akhir match dihitung dan disimpan. Dengan cara ini, Aho-Corasick menjadi sangat efisien untuk pencarian multi-pattern karena seluruh keyword dapat diproses secara bersamaan.

4.1.4. Implementasi Rabin Karp

```

export function searchRabinKarp(

```

```

keywords: string[],
text: string,
sourceType: MatchSource,
caseInsensitive: boolean,
): AlgorithmResult {
    const startTime = performance.now();
    const matches: MatchResult[] = [];
    let comparisonCount = 0;
    const searchableText = caseInsensitive ? text.toLowerCase() : text;

    for (const keyword of keywords) {
        const searchableKeyword = caseInsensitive ? keyword.toLowerCase() :
keyword;
        const result = searchKeyword(text, searchableText, keyword,
searchableKeyword);

        matches.push(...result.matches);
        comparisonCount += result.comparisonCount;
    }

    return {
        matches,
        executionTimeMs: performance.now() - startTime,
        comparisonCount,
        algorithm: "Rabin-Karp",
        source: sourceType,
    };
}

```

4.1.5. Implementasi Fuzzy Matching

```

export function searchWeightedLevenshtein (
    keywords: string[],
    text: string,
    sourceType: MatchSource,
    caseInsensitive: boolean,
    fuzzyThreshold = 0.7,
) : AlgorithmResult

```

```

{
    const start = performance.now();
    const finRes: MatchResult[] = [];
    const tokens = tokenizeWords(text, caseInsensitive);
    const distanceCache = new Map<string, number>();
    const seenMatches = new Set<string>();
    let comparisonCount = 0;

    for (const keyword of keywords) {
        let cKeyword = keyword;
        if (caseInsensitive)
        {
            cKeyword = keyword.toLowerCase();
        }

        if (cKeyword.length === 0) {
            continue;
        }

        for (const token of tokens) {
            if (!shouldCompareToken(cKeyword, token)) {
                continue;
            }

            comparisonCount += 1;
            const distance = getCachedDistance(
                distanceCache,
                cKeyword,
                token.normalizedText,
            );
            const similarity =
                1 - distance / Math.max(cKeyword.length,
token.normalizedText.length);

            if (similarity < fuzzyThreshold) {
                continue;
            }

            const matchKey =

```

```

`${keyword}\n${token.startIndex}\n${token.endIndex}`;
    if (seenMatches.has(matchKey)) {
        continue;
    }

    seenMatches.add(matchKey);
    finRes.push(
        matchResultGenerator(
            keyword,
            token.text,
            token.startIndex,
            token.endIndex,
            distance,
            similarity,
        )
    );
}

const end = performance.now();
const algoRes: AlgorithmResult = (
    algorithmResultGenerator
    (
        finRes,
        end - start,
        comparisonCount,
        "Weighted-Levenshtein",
        sourceType
    )
)
return algoRes;
}

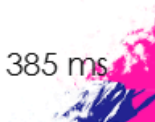
```

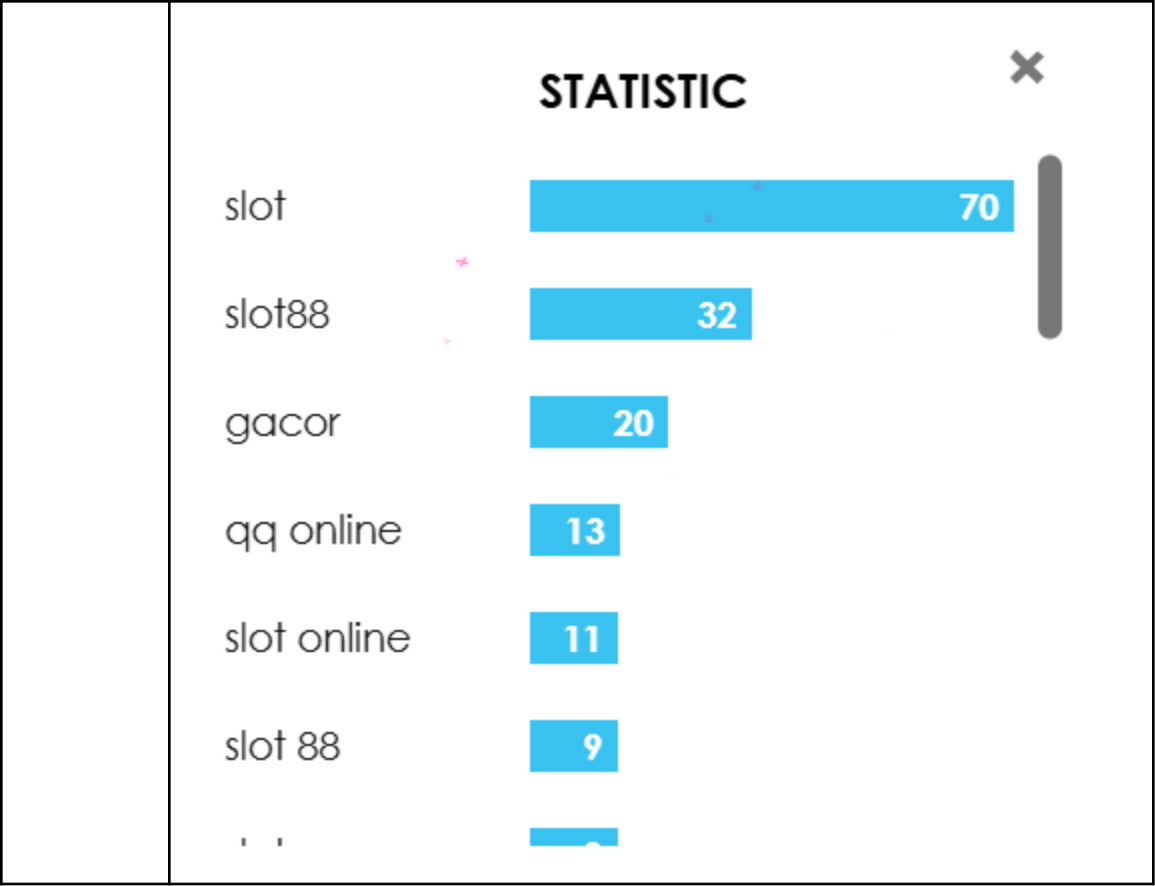
Implementasi Weighted Levenshtein pada repository ini digunakan untuk melakukan fuzzy matching, yaitu mendeteksi keyword yang tidak sama persis tetapi masih memiliki kemiripan tinggi dengan keyword asli. Algoritma ini berguna untuk mengenali kata yang sengaja dimodifikasi, misalnya mengganti huruf dengan angka, simbol, atau karakter yang mirip secara visual seperti slot menjadi sl0t, gacor menjadi g@c0r, atau penggunaan huruf Yunani yang bentuknya menyerupai huruf Latin. Berbeda dengan Levenshtein biasa yang umumnya memberikan biaya edit bernilai sama untuk operasi insert, delete, dan substitute, implementasi Weighted Levenshtein pada repository ini memberikan bobot yang

berbeda untuk setiap jenis perubahan karakter. Misalnya, perubahan karena perbedaan huruf besar dan kecil diberi biaya rendah, karakter dengan aksen diberi biaya rendah, karakter yang mirip secara visual seperti o dan 0 diberi biaya lebih kecil daripada substitusi karakter biasa, sedangkan penyisipan atau penghapusan huruf biasa diberi biaya lebih besar. Model biaya ini dibangun melalui cost model yang berisi aturan untuk berbagai kelompok karakter, termasuk huruf, angka, simbol, whitespace, dan karakter Unicode seperti alfabet Yunani.

4.2. Pengujian

4.2.1. Pengujian 1

Input Web	https://pexl.princeton.edu/															
Result	 Status: Scan complete with warnings Current page: 12NAGA: Situs Slot Gacor Terper... Note: OCR timed out after 20 seconds. Rescan Clear Summary Total keywords found: 24 Total matches: 308 Execution total: 45039 ms Matches <table><tr><td>KMP / DOM Text</td><td>133 matches</td><td>49 ms</td></tr><tr><td>Boyer-Moore / DOM Text</td><td>133 matches</td><td>22 ms</td></tr><tr><td>Regex / DOM Text</td><td>2 matches</td><td>6.2 ms</td></tr><tr><td>Weighted-Levenshtein / DOM Text</td><td>173 matches</td><td>24522 ms</td></tr><tr><td>Rabin-Karp / DOM Text</td><td>133 matches</td><td>385 ms</td></tr></table> 	KMP / DOM Text	133 matches	49 ms	Boyer-Moore / DOM Text	133 matches	22 ms	Regex / DOM Text	2 matches	6.2 ms	Weighted-Levenshtein / DOM Text	173 matches	24522 ms	Rabin-Karp / DOM Text	133 matches	385 ms
KMP / DOM Text	133 matches	49 ms														
Boyer-Moore / DOM Text	133 matches	22 ms														
Regex / DOM Text	2 matches	6.2 ms														
Weighted-Levenshtein / DOM Text	173 matches	24522 ms														
Rabin-Karp / DOM Text	133 matches	385 ms														



	STATISTIC	
member vip	6	
parlay	6	
mahjong ways	5	
pg soft	5	
slot777	5	
sweet bona...	5	
mahjong w...	4	

STATISTIC

maxwin

4

scatter

4

slot gacor te...

4

slot online g...

4

live casino

3

pragmatic p...

3

PRAGMATIC PLAY

HOT GAMES

SLOTS

LIVE CASINO

RACE

TOGEL

OLAH RAGA

CRASH GAME

W

G

GAME EKSKLUSIF DARI HACKSAW!

Status: Scan complete with warnings

Current page: 12NAGA: Situs Slot Gacor Terper...

Note: OCR timed out after 20 seconds

Rescan

Clear

Summary

Total keywords found: 24

Total matches: 309

Execution total: 41850 ms

Matches

KMP / Text 133 matches 53 ms

Boyer-Moore / Text 133 matches 23 ms

Regex / Text 2 matches 5.6 ms

Weighted-Levenshtein / Text 174 matches 21327 ms

Rabin-Karp / Text 133 matches 378 ms

Settings

Highlight: ☐

Blurred text: ☐

OCR image detection: ☐

Rabin-Karp: ☐

Aho-Corasick: ☐

Sound: ☐

Pemberitahuan

menonton main atas ketidakyamanan yang mungkin ditimbulkan. Pemeliharaan Terhadap Gpus pada 18 Des 2025 per 00.00.00

Sen, 18-Mai-2026 13.10.39

JACKPOT PLAY

10R 11,950,494,624

Nex4D Pools Day

2882

Live Draw Min, 17-Mai-2026 00.00.00

GAME POPULER

10R

10R

10R

10R

10R

10R

10R

10R

10R

10R

4.2.2. Pengujian 2

Input Web

https://menu.mamitacomarietta.com/

Result

Current page: JAGO777: Link APK Slot Gacor O...

Rescan

Clear

Summary

Total keywords found: 14

Total matches: 25

Execution total: 2969 ms

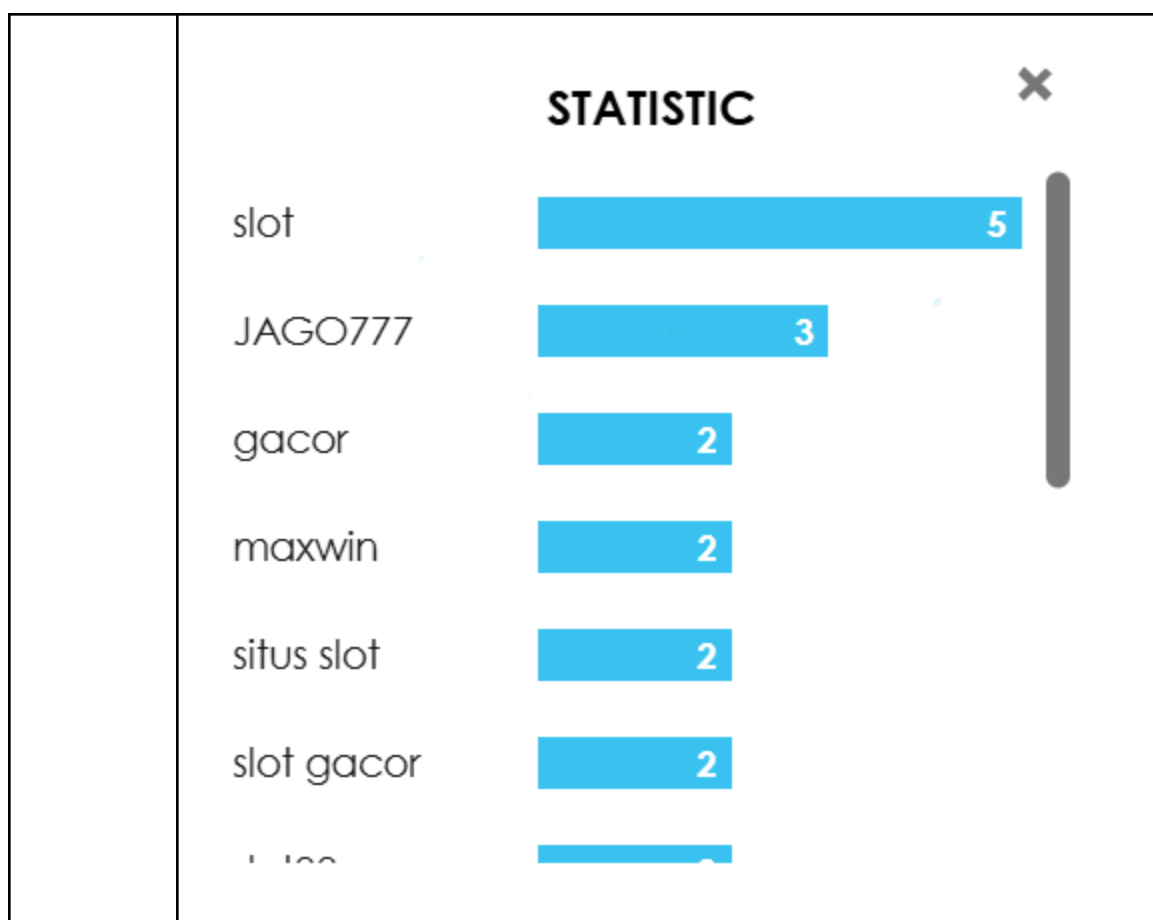
Matches

KMP / DOM Text	9 matches	3.8 ms
Boyer-Moore / DOM Text	9 matches	2.2 ms
Regex / DOM Text	5 matches	1.5 ms
Weighted-Levenshtein / DOM Text	4 matches	289 ms
Rabin-Karp / DOM Text	9 matches	7.9 ms
KMP / Image OCR	2 matches	4.5 ms
Boyer-Moore / Image OCR	2 matches	1.6 ms
Regex / Image OCR	0 matches	5.4 ms
Weighted-Levenshtein / Image OCR	5 matches	236 ms
Rabin-Karp / Image OCR	2 matches	4.3 ms

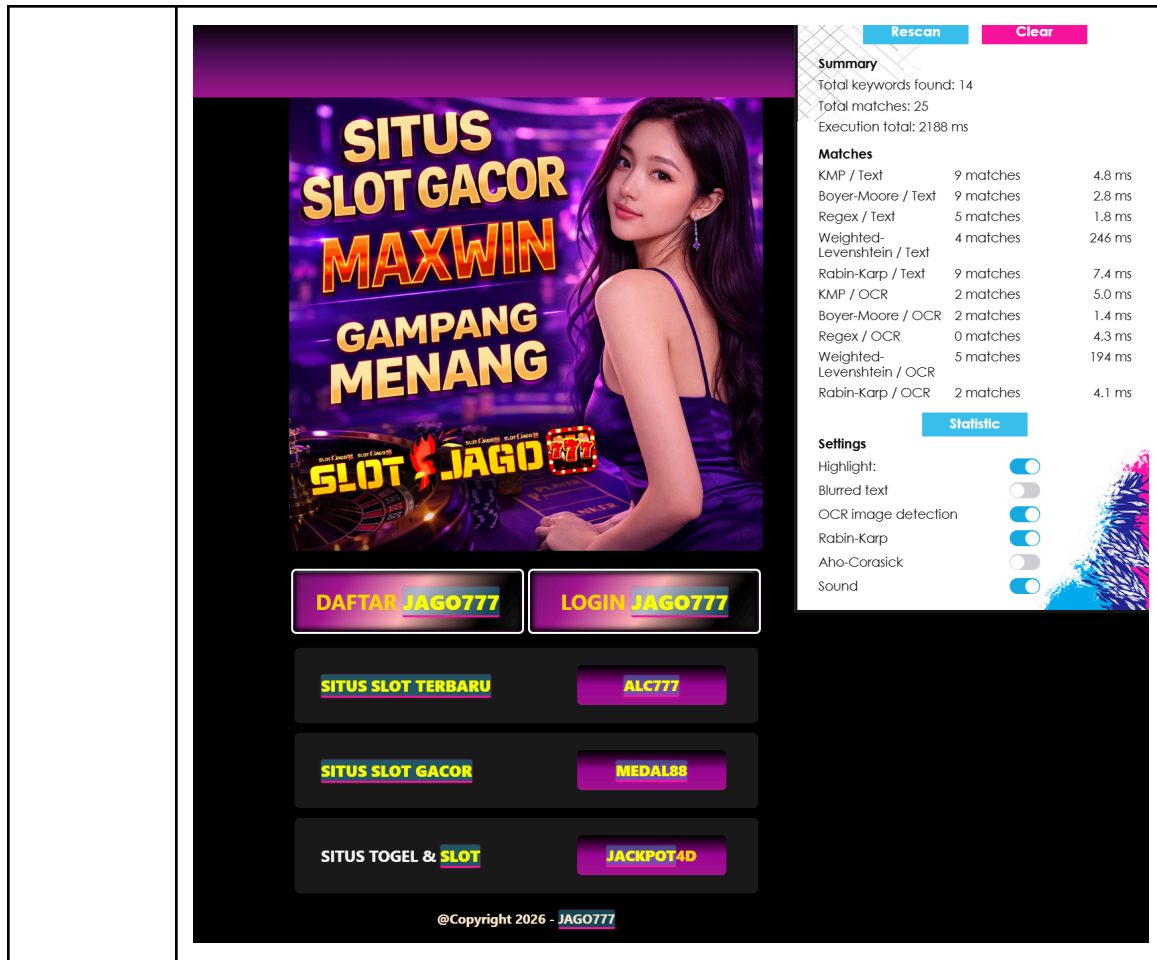
Statistic

Settings

Highlight:



	STATISTIC	
gacor	1	
ALC777	1	
jackpot	1	
judi gacor	1	
MEDAL88	1	
pola gacor	1	
slot terbaru	1	



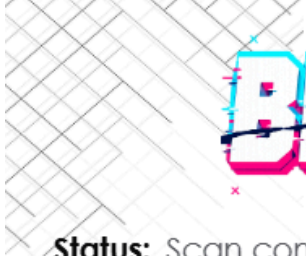
4.2.3. Pengujian 3

Input Web	<pre><!doctype html> <html lang="id"> <head> <meta charset="utf-8" /> <meta name="viewport" content="width=device-width, initial-scale=1" /> <title>TC01 Exact Multi Keyword</title> <style> :root { font-family: Arial, Helvetica, sans-serif; line-height: 1.5; background: #f7f7f7; color: #222; } body { margin: 0; padding: 24px; }</pre>
-----------	--

	<pre> main { max-width: 900px; margin: 0 auto; background: white; border: 1px solid #ddd; border-radius: 12px; padding: 24px; } .card { border: 1px solid #ddd; border-radius: 10px; padding: 16px; margin: 14px 0; } .badge { display: inline-block; border: 1px solid #aaa; border-radius: 999px; padding: 2px 10px; margin-right: 6px; font-size: 0.9rem; } .note { color: #666; font-size: 0.9rem; } </style> </head> <body> <!-- TC01: Exact matching. Target: KMP, Boyer-Moore, Aho-Corasick, Rabin-Karp. Catatan: halaman ini sengaja memakai keyword dari keyword.txt dalam bentuk normal. --> <main> <h1>TC01 - Exact Multi Keyword</h1> <p class="note">Gunakan halaman ini untuk menguji pencarian kata kunci biasa tanpa obfuscation.</p> <section class="card"> <h2>Banner artikel</h2> <p> Di sebuah halaman palsu, muncul frasa slot gacor hari ini, jackpot maxwin, scatter merah, dan rtp live secara berulang dalam paragraf promosi. Beberapa baris juga menyebut slot online terpercaya no 1 dan situs slot gacor terpercaya. </p> </section> </pre>
--	---

	<pre> <section class="card"> <h2>Promo dan transaksi</h2> <p> Pengunjung diarahkan untuk mengambil bonus new member, bonus deposit, bonus cashback, free spin, deposit dana, deposit qris, deposit pulsa, serta wd cepat setelah membuat akun. </p> </section> <section class="card"> <h2>Provider dan game</h2> <p> Konten lain menampilkan pragmatic play, pg soft, habanero, joker gaming, microgaming, spadegaming, mahjong ways 2, sweet bonanza, starlight princess, gates of olympus, dan wild west gold. </p> </section> <section class="card"> <h2>Variasi kapitalisasi</h2> <p> SLOT GACOR, Judi Online, Casino Online, LIVE CASINO, dan RTP TERTINGGI ditulis dalam huruf besar untuk menguji mode case-insensitive. </p> </section> </main> </body> </html> </pre>
--	--

Result



Status: Scan complete
Current page: TC01 Exact Multi Keyword

[Rescan](#) [Clear](#)

Summary
Total keywords found: 24
Total matches: 93
Execution total: 2309 ms

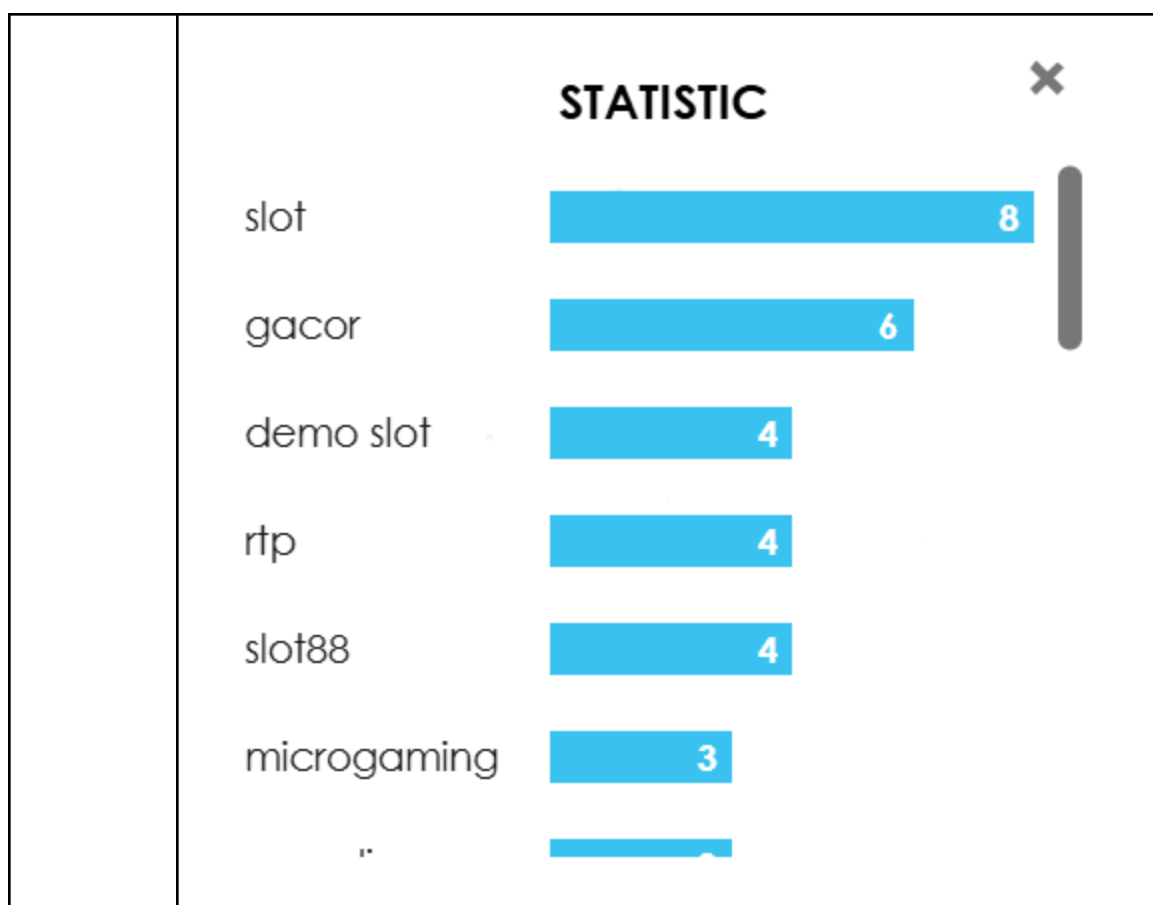
Matches

KMP / DOM Text	48 matches	9.9 ms
Boyer-Moore / DOM Text	48 matches	5.5 ms
Regex / DOM Text	1 matches	1.6 ms
Weighted-Levenshtein / DOM Text	44 matches	2245 ms
Rabin-Karp / DOM Text	48 matches	38 ms

[Statistic](#)

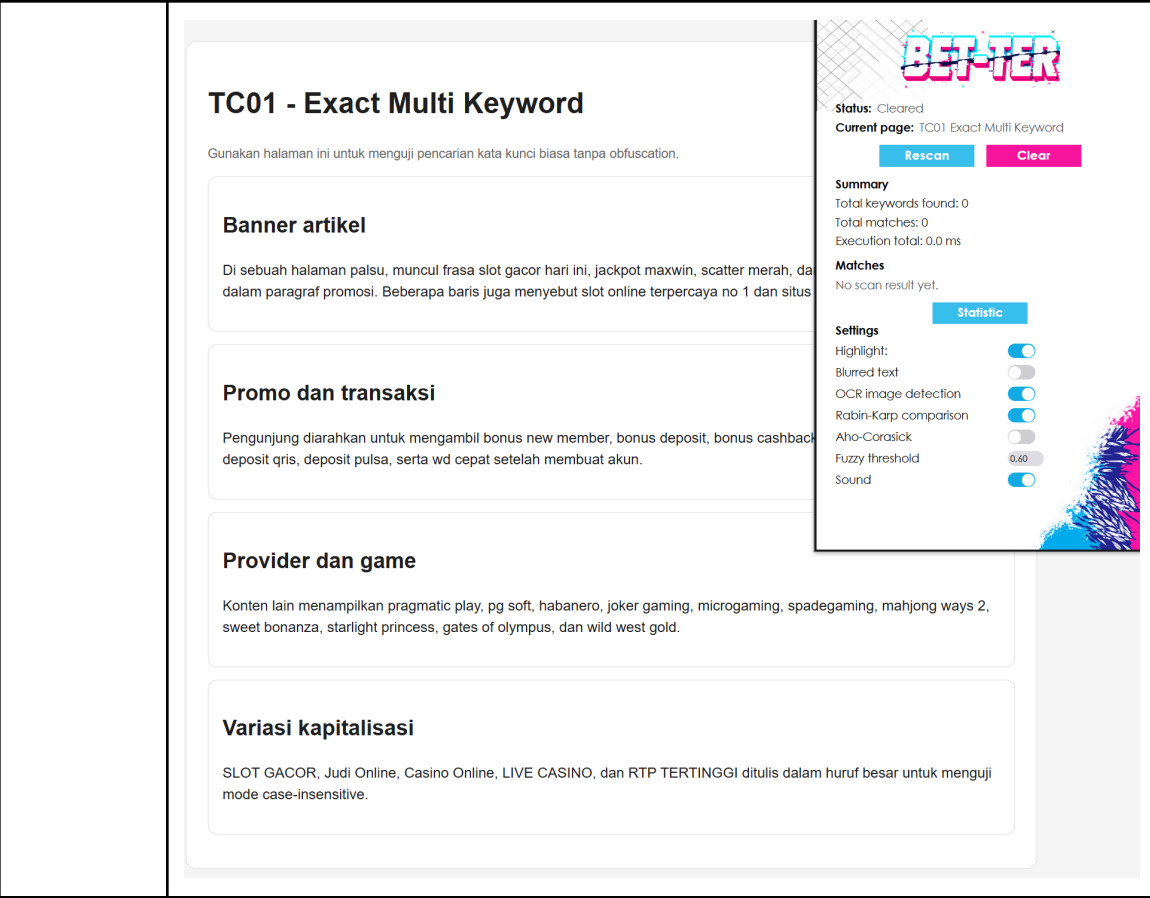
Settings





	STATISTIC	
deposit dana	2	
deposit qris	2	
habanero	2	
jackpot	2	
joker gaming	2	
maxwin	2	
parlay	2	

	STATISTIC	
wd cepat	2	
agen terper...	1	
anti kalah	1	
bonanza slot	1	
bonus cash...	1	
bonus deposit	1	



4.2.4. Pengujian 4

Input Web	<pre><!doctype html> <html lang="id"> <head> <meta charset="utf-8" /> <meta name="viewport" content="width=device-width, initial-scale=1" /> <title>TC02 Regex Digit Suffix</title> </style> :root { font-family: Arial, Helvetica, sans-serif; line-height: 1.5; background: #f7f7f7; color: #222; } body { margin: 0; padding: 24px; } main { max-width: 900px;</pre>
-----------	---

	<pre> margin: 0 auto; background: white; border: 1px solid #ddd; border-radius: 12px; padding: 24px; } .card { border: 1px solid #ddd; border-radius: 10px; padding: 16px; margin: 14px 0; } .badge { display: inline-block; border: 1px solid #aaa; border-radius: 999px; padding: 2px 10px; margin-right: 6px; font-size: 0.9rem; } .note { color: #666; font-size: 0.9rem; } </style> </head> <body> <!-- TC02: RegEx matching. Target utama: pola <kata><2 atau 3 angka> tanpa spasi. Target tambahan: exact algorithms tetap bisa menemukan keyword dasar di sekitar teks. Edge case sengaja disisipkan: satu angka, empat angka, spasi sebelum angka, tanda baca. --> <main> <h1>TC02 - Regex Digit Suffix</h1> <p class="note">Gunakan halaman ini untuk menguji pola kata diikuti 2/3 angka tanpa spasi.</p> <section class="card"> <h2>Pola positif</h2> <p> Iklan palsu menampilkan MAXWIN88, SLOT99, GACOR777, JACKPOT123, HOKI88, MADU308, RTP88, ZEUS99, BONUS200, SCATTER777, dan CASINO123. </p> <p> Variasi tanda baca: MAXWIN234! SLOT88, GACOR99. JACKPOT777? </p> </section> </body> </pre>
--	---

	<pre> bonus100; rtp777: zeus88. </p> </section> <section class="card"> <h2>Edge case regex</h2> <p> Bentuk yang seharusnya diuji hati-hati: MAXWIN tanpa angka, SLOT 99 dengan spasi, GACOR9 dengan satu angka, GACOR9999 dengan empat angka, dan OLYMPUS1000 dengan empat angka. </p> <p> Kata lain di sekitar halaman tetap memuat slot gacor, jackpot, bonus deposit, dan deposit ewallet untuk pembanding exact matching. </p> </section> <section class="card"> <h2>Campuran DOM</h2> <button type="button">CLAIM BONUS100</button> SLOT88 MAXWIN234 RTP777 </section> </main> </body> </html> </pre>
--	---

Result



Status: Scan complete
Current page: TC02 Regex Digit Suffix

[Rescan](#) [Clear](#)

Summary
Total keywords found: 24
Total matches: 102
Execution total: 1820 ms

Matches

KMP / DOM Text	28 matches	8.3 ms
Boyer-Moore / DOM Text	28 matches	2.7 ms
Regex / DOM Text	25 matches	0.0 ms
Weighted-Levenshtein / DOM Text	49 matches	1778 ms
Rabin-Karp / DOM Text	28 matches	28 ms

[Statistic](#)

Settings
Highlight: ☒
Blurred text: ☐





	STATISTIC	
MAXWIN234	2	
olympus1000	2	
slot 777	2	
slot 88	2	
slot cq9	2	
slot ion	2	
slot jdb	2	

STATISTIC

slot vip

2

slot xl

2

slot777

2

SLOT88

2

bonus deposit

1

bonus100

1

TC02 - Regex Digit Suffix

Gunakan halaman ini untuk menguji pola kata diikuti 2/3 angka tanpa spasi.

Pola positif

Iklan palsu menampilkan MAXWIN88, SLOT99, GACOR777, JACKPOT123, HOKI88, MADU BONUS200, SCATTER777, dan CASINO123.

Variasi tanda baca: MAXWIN234! SLOT88, GACOR99, JACKPOT777? bonus100; rtp777: ze

Edge case regex

Bentuk yang seharusnya diuji hati-hati: MAXWIN tanpa angka, SLOT 99 dengan spasi, GACOR99999 dengan empat angka, dan OLYMPUS1000 dengan empat angka.

Kata lain di sekitar halaman tetap memuat slot gacor, jackpot, bonus deposit, dan deposit ewa matching.

Campuran DOM

CLAIM BONUS100SLOT88MAXWIN234RTP777

BETTER

Status: Scan complete

Current page: TC02 Regex Digit Suffix

RescanClear

Summary

Total keywords found: 24

Total matches: 102

Execution total: 1873 ms

Matches

KMP / DOM Text	28 matches	13 ms
Boyer-Moore / DOM Text	28 matches	2.4 ms
Regex / DOM Text	25 matches	1.8 ms
Weighted-Levenshtein / DOM Text	49 matches	1826 ms
Rabin-Karp / DOM Text	28 matches	27 ms

Settings

Highlight: ☒

Blurred text: ☐

OCR image detection: ☐

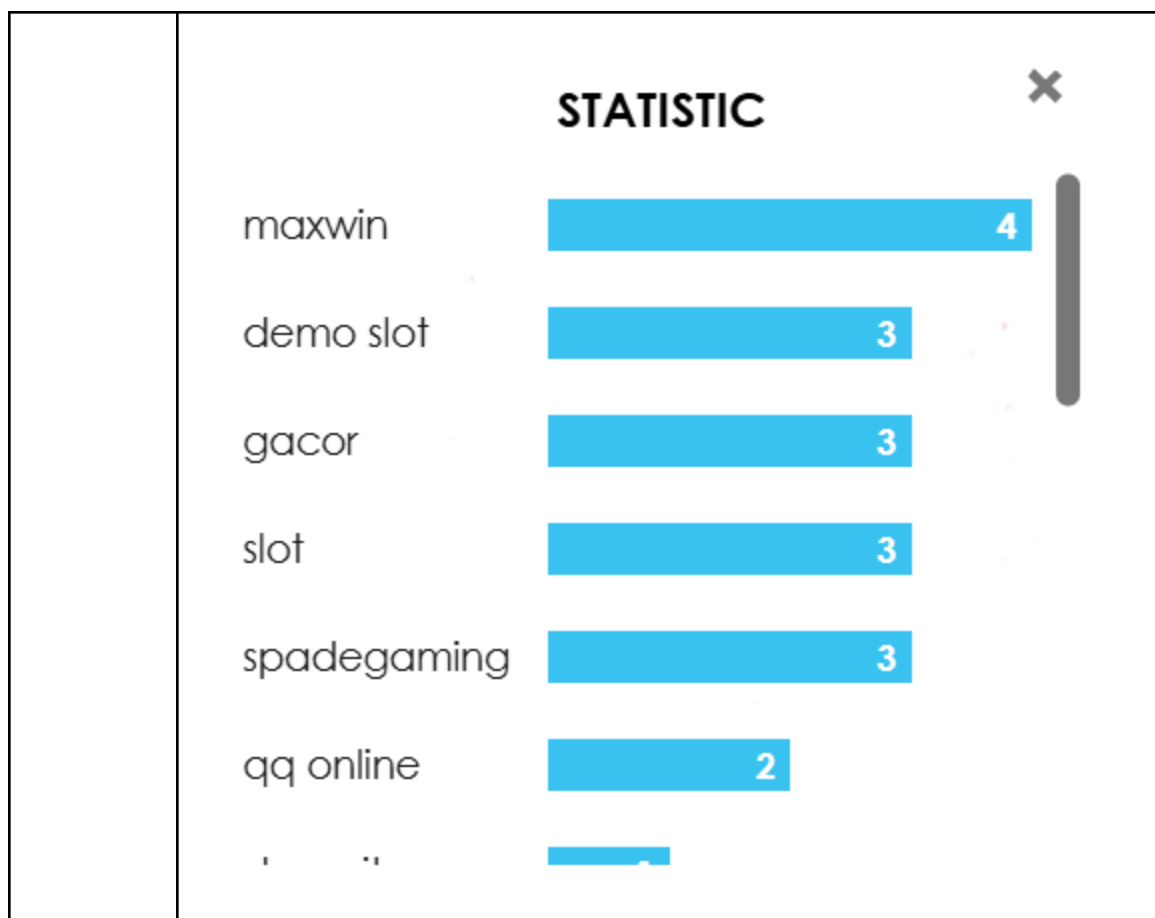
Rabin-Karp comparison: ☐

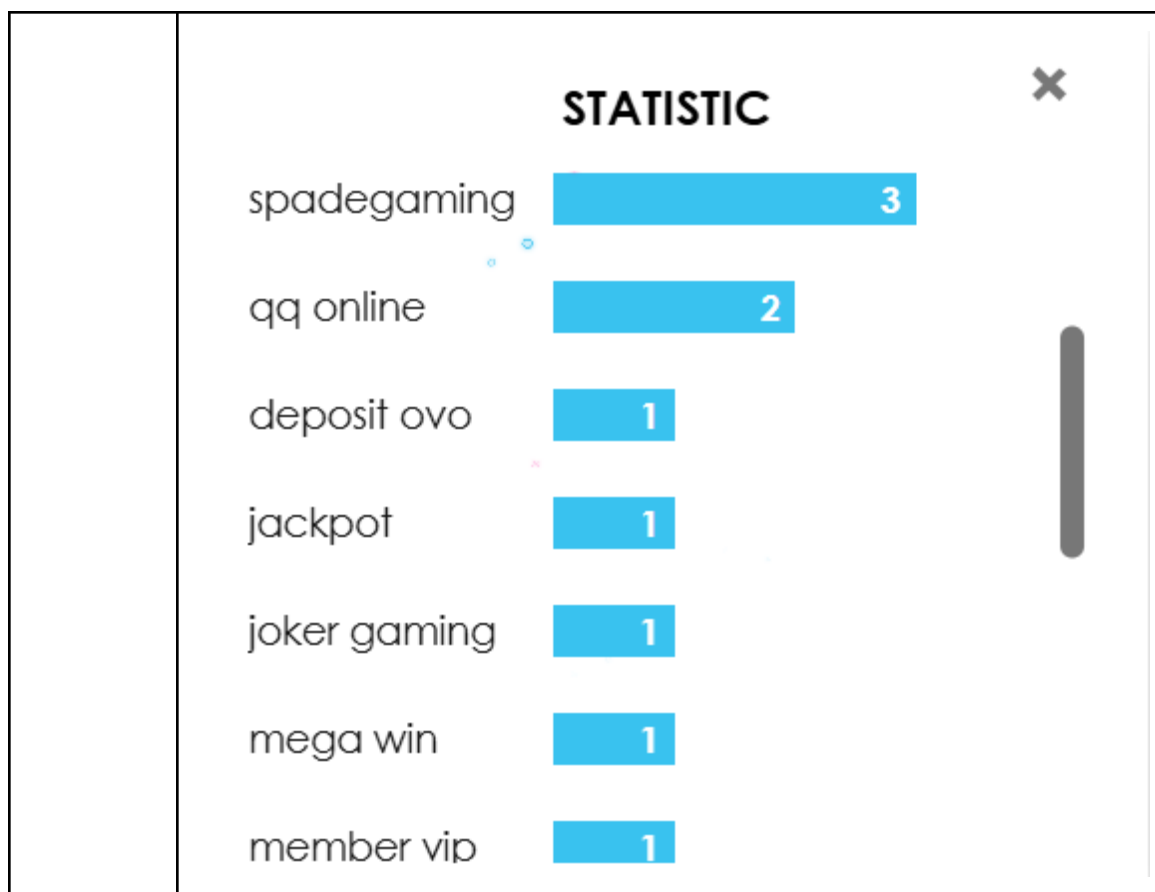
48

4.2.5. Pengujian 5

Input Web	<pre><!doctype html> <html lang="id"> <head> <meta charset="utf-8" /> <meta name="viewport" content="width=device-width, initial-scale=1" /> <title>TC03 Fuzzy Unicode Leetspeak</title> <style> .root { font-family: Arial, Helvetica, sans-serif; line-height: 1.5; background: #f7f7f7; color: #222; } body { margin: 0; padding: 24px; } main { max-width: 900px; margin: 0 auto; background: white; border: 1px solid #ddd; border-radius: 12px; padding: 24px; } .card { border: 1px solid #ddd; border-radius: 10px; padding: 16px; margin: 14px 0; } .badge { display: inline-block; border: 1px solid #aaa; border-radius: 999px; padding: 2px 10px; margin-right: 6px; font-size: 0.9rem; } .note { color: #666; font-size: 0.9rem; } </style> </head> <body></pre>
-----------	---

	<pre> <!-- TC03: Fuzzy matching / Weighted Levenshtein. Target: homoglyph unicode, alphabet Greek, angka mirip huruf, dan typo kecil. Contoh substitusi: a -> α, o -> o/0, i -> 1/i, e -> 3, u -> v/7. --> <main> <h1>TC03 - Fuzzy Unicode dan Leetspeak</h1> <p class="note">Gunakan halaman ini untuk menguji pencocokan mirip visual dan manipulasi karakter.</p> <section class="card"> <h2>Greek homoglyph</h2> <p> Frasa tersamar: slot gacor, maxwin, scatter merah, jackpot maxwin, casino online, pragmatic play, mahjong ways, gates of olympus, starlight princess, dan sweet bonanza. </p> <p> Huruf α adalah Greek alpha, sedangkan o adalah Greek omicron, bukan huruf latin biasa. </p> </section> <section class="card"> <h2>Leetspeak angka</h2> <p> Variasi lain: sl0t gac0r, g4c0r, maxw1n, b0nus new m3mber, b0n7s dep0sit, dep0sit d4na, dep0sit qris, wd c3pat, jud1 Online, sltus jud1, dan c7an maxw1n. </p> </section> <section class="card"> <h2>Typo kecil</h2> <p> Bentuk salah ketik ringan: pragmat1c play, mcirogaming, spadegamming, starligth princess, mahj0ng wayz, gates of olympuz, dan bonanza sl0t. </p> </section> <section class="card"> <h2>Kontrol negatif</h2> <p> Alpha beta gamma delta epsilon adalah nama huruf Yunani biasa. Angka 2026 dan kata algoritma, browser, extension, serta laporan seharusnya tidak dianggap sebagai konten target. </p> </section> </pre>
--	---





	STATISTIC	
...		
scatter	1	
slot 88	1	
slot spadeg...	1	
slot xl	1	
slot88	1	
TC03	1	

TC03 - Fuzzy Unicode dan Leetspeak

Gunakan halaman ini untuk menguji pencocokan mirip visual dan manipulasi karakter.

Greek homoglyph

Frasa tersamar: slot gacor, maxwin, scatter merah, jackpot maxwin, casino online, pragmatic, of olympus, starlight princess, dan sweet bonanza.

Huruf α adalah Greek alpha, sedangkan o adalah Greek omicron, bukan huruf latin biasa.

Leetspeak angka

Variasi lain: sl0t gac0r, g4c0r, maxw1n, b0nus new m3mber, b0n7s dep0sit, dep0sit d4na, dep0 Online, s1tus jud1, dan c7an maxw1n.

Typo kecil

Bentuk salah ketik ringan: pragmat1c p1ay, mcirogaming, spadegamming, starligh princess, mahj0ng wayz, gates of olympuz, dan bonanza sl0t.

Kontrol negatif

Alpha beta gamma delta epsilon adalah nama huruf Yunani biasa. Angka 2026 dan kata algoritma, browser, extension, serta laporan seharusnya tidak dianggap sebagai konten target.

BET-TER

Status: Scanning...

Current page: TC03 Fuzzy Unicode Leetspeak

ScanningClear

Summary

Total keywords found: 0

Total matches: 0

Execution total: 0.0 ms

Matches

Scanning current page...

Statistic

Settings

Highlight: ☐

Blurred text: ☐

OCR image detection: ☐

Rabin-Karp comparison: ☐

Aho-Corasick: ☐

Fuzzy threshold: 0.60

Sound: ☐

55

BAB 5

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil implementasi dan pengujian, dapat disimpulkan bahwa terdapat banyak algoritma yang dapat digunakan untuk pattern matching, seperti KMP, Boyer-Moore, RegEx, Aho-Corasick, Rabin-Karp, dan Weighted Levenshtein Distance. Pada Judol Detector, kombinasi KMP, Boyer-Moore, RegEx, dan Weighted Levenshtein Distance membuat sistem mampu mendeteksi kata kunci secara persis, pola <kata><angka>, serta variasi kata yang disamarkan. Dengan adanya fitur highlight, tooltip, dan statistik realtime, hasil deteksi juga dapat ditampilkan secara lebih jelas kepada pengguna.

5.2. Saran

Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan menambahkan algoritma lain seperti Aho-Corasick atau Rabin-Karp sebagai pembanding performa. Selain itu, normalisasi teks dan threshold pada Weighted Levenshtein Distance dapat diperbaiki agar deteksi lebih akurat dan mengurangi *false positive*. Fitur tambahan seperti blur teks dan OCR gambar juga dapat dikembangkan agar sistem mampu mendeteksi konten judi online dalam bentuk yang lebih beragam.

LAMPIRAN

- Link Repository Github: https://github.com/StevenT-1/Tubes3_Bet-ter/
- Link Video:
<https://drive.google.com/drive/folders/1CoxmJgg3GFL-KHJ1oE-R4q1ZsfZ7ceZX?usp=sharing>
- Tabel Checklist Spesifikasi

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex handle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video	✓	
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

- Tabel Pembagian Tugas:
https://docs.google.com/document/d/1b_qMZvalrdhtr-xIQ8YJywwIDjgnJ4J-WgdnSxS2MD8/edit?usp=sharing

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, appearing to read 'Ishak' with a stylized flourish at the end.

Ishak Palentino Napitupulu

A handwritten signature in brown ink, appearing to read 'Steven' with a stylized flourish at the end.

Steven Tan

A handwritten signature in black ink, appearing to read 'Marcel' with a stylized flourish at the end.

Marcel Luther Sitorus

DAFTAR PUSTAKA

- [1] Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, "[Spesifikasi Tugas Besar 3 - IF2211 - 2025/2026](#)" 2026. [Online]. Tersedia: Google Docs. Diakses: 30 Mei 2026.
- [2] TutorialsPoint, "TypeScript Tutorial," TutorialsPoint. [Online]. Tersedia: TutorialsPoint TypeScript Tutorial. Diakses: 24 Mei 2026.
- [3] Wikipedia contributors, "Levenshtein distance," Wikipedia, The Free Encyclopedia. [Online]. Tersedia: Wikipedia. Diakses: 38 Mei 2026.
- [4] OpenGenus IQ contributors, "Aho Corasick Algorithm," OpenGenus IQ, [Online]. Tersedia: <https://iq.opengenus.org/aho-corasick-algorithm/>. Diakses: 27 Mei 2026.